Assignment: LU decomposition and Matrix Inverse

Academic Integrity Declaration

Check the statement that applies, and sign your name.

Check	Statement
V	I have **NOT** used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment.
	I have **NOT completed this assignment entirely on my own**.

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

Examples:

- Screenshots of LLM prompt,
- Explanation of assistance : who/how/what extent etc

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.10.30

Sign: 유채은

What you need to submit

Submit as a zip file: **Assignment_LUdecomp_YuChaeeun_22200476.zip**

- **Report** (pdf)
- **Src Codes:** **(1) Assignment_ LUdecomp _YuChaeeun_22200476.cpp**
  (2) myNP_22200476.h, myNP_22200476.cpp
  (3) myMatrix_22200476.h, myMatrix _22200476.cpp

- Review how to use structure type for using 2D array
- Review how to use matrix structure, (Tutorial Video)
- You should insert exceptional/error handling
  e.g. giving error message when square matrix is not used, div by zero etc.
  *We will only consider square matrix A (n by n) for this assignment.

Datafile:

[Directory]

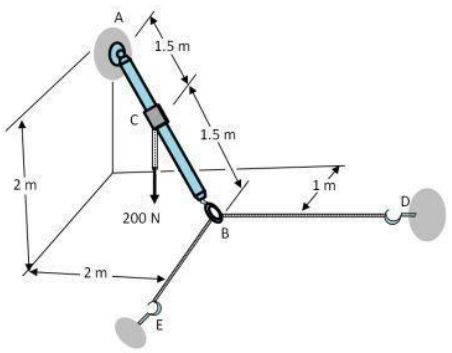
\NP\NP_Data \Assignment6\

[File Name] (*.txt)

Question No.	Variable Name	File Name
Q1	matrix A vector b	prob1_matA.txt prob1_vecb.txt
Q2	matrix A vector b	prob2_matK.txt prob2_vecf.txt

Problem: Solve the following linear systems

Q1. Determine the reactions at the ball joint and tension forces in the following system.



$$\sum \mathbf{F} = 0, \sum \mathbf{M}_A = 0$$

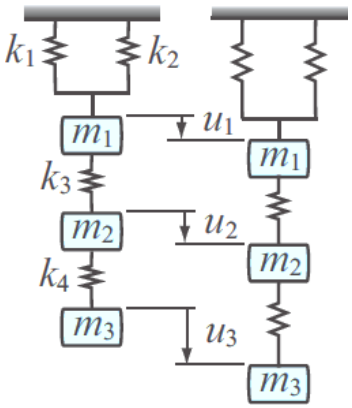
$$A_x + T_E = 0$$
$$A_y + T_D = 0$$
$$A_z - 200 = 0$$
$$T_D - 2T_E = 0$$
$$-2T_E + 100 = 0$$

ANS: Ax= -50
Ay= -100.00
Az= 200.00
TD= 100.00
TE= 50.00

Q2. Determine the displacement (u_1, u_2, u_3) from Ku=f

They are in the equilibrium states, and u_1, u_2, u_3 are the relative displacement for each mass.

m_1=2 kg, m_2=3kg, m_3=1.5kg , g=9.81 m/s^2
k_1= 30 [N/m], k_2=25 [N/m], k_3=20 [N/m], k_4=15 [N/m]



$$(k_1 + k_2 + k_3)u_1 - k_3u_2 = m_1g$$
$$-k_3u_1 + (k_3 + k_4)u_2 - k_4u_3 = m_2g$$
$$-k_4u_2 + k_4u_3 = m_3g$$

ANS: u_1=1.159364 u_2= 3.366614 u_3= 4.347614

Procedure

1. LU decomposition without partial pivoting [20pt].

If scaled partial pivoting is applied [extra +10 pt]

a) Write a pseudocode for LU decomposition [5pt]

// pseudocode goes here

```
// find the biggest fabs(element) in that column
big = k
max = | U[k][k] |
for i = k to m
        if ( | U[i][k] | > max)
                big = i
                max = | U[i][k] |
end i

// row swapping (row k <-> row big)
for j = 0 to n
        temp = P[k][j]
        tmpU = U[k][j]
        tmpL = L[k][j]

        P[k][j] = P[big][j]
        U[k][j] = U[big][j]
        L[k][j] = L[big][j]

        P[big][j] = temp
        U[big][j] = tmpU
        L[big][j] = tmpL
End j

// LU decomposition (row reduction)
for i = k + 1 to m

        mult = U[i][k] / U[k][k]
        L[i][k] = mult;
        for j = k to n
                U[i][j] = U[i][j] - mult * U[k][j]

        End j
End i
```

---

b) Create a C function that processes the LU decomposition [15pt]

- Input: matrix **A**(n x n), vector **b**(n x1)
- Output: matrix **U**, matrix L (option)permutation matrix **P**

```
void LUdecomp (Matrix A, Matrix L, Matrix U);
```

For Pivoting(extra)

```
void LUdecomp (Matrix A, Matrix L, Matrix U, Matrix P);    // for pivoting
```

```
void LUdecomp(Matrix* A, Matrix* L, Matrix* U, Matrix* P)  // -------> divide A into L & U
{
	// initialization
	*U = copyMat(*A);
	int m = U->rows;
	int n = U->cols;
	double mult = 0;
	int big;
	double max, temp, tmpU, tmpL = 0;


	for (int k = 0; k < (m - 1); k++) {

		printf("k = %d\n", k);

		// find the biggest fabs(element) in that column
		big = k;
		max = fabs(U->at[k][k]);
		for (int i = k; i < m; i++) {
			if (fabs(U->at[i][k]) > max) {
				big = i;
				max = fabs(U->at[i][k]);

			}
		}
		// row swapping (row k <-> row big)
		for (int j = 0; j < n; j++) {
			temp = P->at[k][j];
			tmpU = U->at[k][j];
			tmpL = L->at[k][j];

			P->at[k][j] = P->at[big][j];
			U->at[k][j] = U->at[big][j];
			L->at[k][j] = L->at[big][j];

			P->at[big][j] = temp;
			U->at[big][j] = tmpU;
			L->at[big][j] = tmpL;

		} /*END row swapping*/


		// LU decomposition (row reduction)
		for (int i = k + 1; i < m; i++) {

			mult = U->at[i][k] / U->at[k][k];
```

```
                        L->at[i][k] = mult;
                        for (int j = k; j < n; j++) {
                                U->at[i][j] = U->at[i][j] - mult * U->at[k][j];
                }

                }/*END row-reduction*/

        }


}
```

2. Create a function that solves for Ax= LUx=b [20pt]

If permutation P is applied [extra +10pt]

a) Write a pseudocode to solve Ax=b using LU decomposition [5pt]

```
// initialize & step 1. LU factorization
        Pb = P * b
        y = U * x

// step 2. Ly=b fwdsub
        for i = 0 to (m-1)
                sum = 0

                for j = 0 to (i - 1)
                        sum += L[i][j] * y[j]
                end j
                y[i] = Pb[i] - sum;
        end i
```

```
// step 3. Ux=y bcksub
        for i = m – 1 to 0
                sum = 0
                for j = i + 1 to n-1
                        sum += U[i][j] * x[j]
                end j
                x[i] = (y[i][0] - sum) / U[i][i]
        end i
```

---

b) Create C-program functions [15pt]

```
void solveLU (Matrix L, Matrix U, Matrix b, Matrix x);     // or
void fwdsub (Matrix L, Matrix b, Matrix y);
void backsub (Matrix U, Matrix y, Matrix x);

For Pivoting(extra)
void solveLU (Matrix L, Matrix U, Matrix P, Matrix b, Matrix x);
```

```
void solveLU(Matrix* L, Matrix* U, Matrix* P, Matrix* b, Matrix* x)
{
        // initialize & step 1. LU factorization
        Matrix Pb = createMat(b->rows, 1);
        Matrix y = createMat(b->rows, 1);
        Pb = multMat(*P, *b);
        int m = U->rows;
        int n = U->cols;
```

```
	// step 2. Ly=b fwdsub
	double sum = 0;

	for (int i = 0; i < m; i++) {
		sum = 0;

		for (int j = 0; j < i; j++) {
			sum += L->at[i][j] * y.at[j][0];
		}

		y.at[i][0] = Pb.at[i][0] - sum;
	}



	// step 3. Ux=y bcksub

	for (int i = m - 1; i >= 0; i--) {
		sum = 0;

		for (int j = i + 1; j < n; j++) {
			if (i == m - 1) {
				break;
			}
			else
			sum += U->at[i][j] * x->at[j][0];
		}

		x->at[i][0] = (y.at[i][0] - sum) / U->at[i][i];
	}

}
```

3. Create a function that finds the inverse of A. [20pt]

double invMat(Matrix A, Matrix Ainv);

- MUST check if A is square (nxn) and it is full rank. rank(A)=n
  ➔ Hint: check if there is any zero in diagonal terms of matrix **U**
- Confirm your answer by calculating **x=Ainv*b**

*** For LU with pivoting, check your process with the values shown in Appendix. TA will check your library with test matrices that need pivoting.*

```
// code

double invMat(Matrix* A, Matrix* Ainv)

{
	// initialize
	int m = A->rows;		int n = A->cols;
	Matrix A_ = copyMat(*A);	Matrix L = zeros(m, n);
	Matrix U = zeros(m, n);		Matrix P = eye(m, n);
	Matrix y = zeros(n, 1);		Matrix v = zeros(n, 1);
	Matrix b = zeros(n, 1);

	if (m != n) {
		printf("DIMENSION ERROR!! m != n\n\n");
		return 1;
	}

	// LUdecomp
	LUdecomp(&A_, &L, &U, &P);

	// solve x
	for (int i = 0; i < n; i++) {

		b.at[i][0] = 1;

		fwdsub(&L, &y, &b);
		backsub(&U, &v, &y);

		for (int j = 0; j < n; j++) {
			Ainv->at[j][i] = v.at[j][0];
		}
		b.at[i][0] = 0;
	}
}
```

---

4. Capture the output results to solve Q1 and Q1 [10pt]

Q1.

```
L =
       1.000000        0.000000        0.000000        0.000000        0.000000
       0.000000        1.000000        0.000000        0.000000        0.000000
       0.000000        0.000000        1.000000        0.000000        0.000000
       0.000000        0.000000        0.000000        1.000000        0.000000
       0.000000        0.000000        0.000000        0.000000        1.000000

U =
       1.000000        0.000000        0.000000        0.000000        1.000000
       0.000000        1.000000        0.000000        1.000000        0.000000
       0.000000        0.000000        1.000000        0.000000        0.000000
       0.000000        0.000000        0.000000        1.000000       -2.000000
       0.000000        0.000000        0.000000        0.000000        2.000000

P =
       1.000000        0.000000        0.000000        0.000000        0.000000
       0.000000        1.000000        0.000000        0.000000        0.000000
       0.000000        0.000000        1.000000        0.000000        0.000000
       0.000000        0.000000        0.000000        1.000000        0.000000
       0.000000        0.000000        0.000000        0.000000        1.000000

========= Q2 =========
b =
       0.000000
       0.000000
     200.000000
       0.000000
     100.000000

x =
     -50.000000
    -100.000000
     200.000000
     100.000000
      50.000000
```

Q2.

```
L =
       1.000000         0.000000         0.000000
      -0.266667         1.000000         0.000000
       0.000000        -0.505618         1.000000

U =
      75.000000       -20.000000         0.000000
       0.000000        29.666667       -15.000000
       0.000000         0.000000         7.415730

P =
       1.000000         0.000000         0.000000
       0.000000         1.000000         0.000000
       0.000000         0.000000         1.000000

========= Q2 =========
b =
      19.620001
      29.430000
      14.715000

x =
       1.159364
       3.366614
       4.347614
```

5. Check your answer with the output from MATLAB [5pt]

Write Matlab code

```
// your MATLAB code

[L, U, P] = lu(A);

x = A \ b;
```

// MATLAB output

Q1. ----------

```
x =

   -50
  -100
   200
   100
    50

>> b

b =

     0
     0
   200
     0
   100

>> L

L =

     1     0     0     0     0
     0     1     0     0     0
     0     0     1     0     0
     0     0     0     1     0
     0     0     0     0     1
```

```
U =

     1     0     0     0     1
     0     1     0     1     0
     0     0     1     0     0
     0     0     0     1    -2
     0     0     0     0     2

>> P

P =

     1     0     0     0     0
     0     1     0     0     0
     0     0     1     0     0
     0     0     0     1     0
     0     0     0     0     1
```

Q2. ----------

```
x =

    1.1594
    3.3666
    4.3476

>> b

b =

   19.6200
   29.4300
   14.7150

>> L

L =

    1.0000         0         0
   -0.2667    1.0000         0
         0   -0.5056    1.0000
```

```
U =

   75.0000  -20.0000         0
         0   29.6667  -15.0000
         0         0    7.4157

>> P

P =

     1     0     0
     0     1     0
     0     0     1
```

Appendix

1. Permutation matrix **P f**or LU
   y=fwdsub(**L**, **P***b**) → x=backsub(**U**,**y**)

2. Update the permutation matrix **P** during the elimination process

3. Matrix elements for debugging the process of finding P, L, U using LU decomposition.

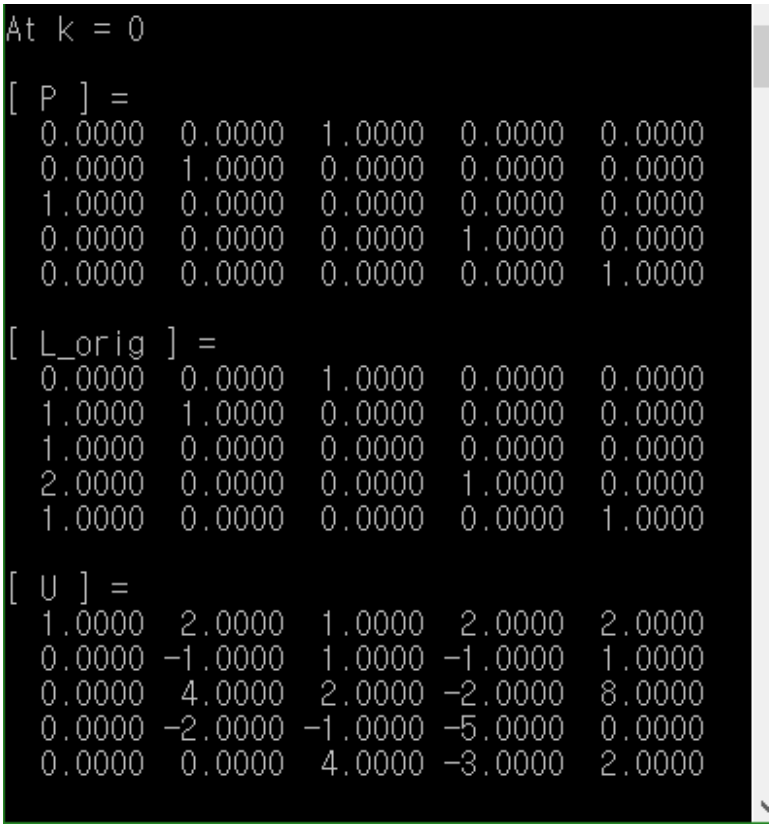
   Example) Matrix A :

```
[ matA ] =
  0.0000   4.0000   2.0000  -2.0000   8.0000
  1.0000   1.0000   2.0000   1.0000   3.0000
  1.0000   2.0000   1.0000   2.0000   2.0000
  2.0000   2.0000   1.0000  -1.0000   4.0000
  1.0000   2.0000   5.0000  -1.0000   4.0000
```

Final Output of P, L, U Matrix :

```
[ P ] =
  0.0000   0.0000   1.0000   0.0000   0.0000
  0.0000   1.0000   0.0000   0.0000   0.0000
  0.0000   0.0000   0.0000   1.0000   0.0000
  1.0000   0.0000   0.0000   0.0000   0.0000
  0.0000   0.0000   0.0000   0.0000   1.0000
[ L ] =
  1.0000   0.0000   0.0000   0.0000   0.0000
  1.0000   1.0000   0.0000   0.0000   0.0000
  2.0000   2.0000   1.0000   0.0000   0.0000
  0.0000  -4.0000  -2.0000   1.0000   0.0000
  1.0000   0.0000  -1.3333   0.5833   1.0000
[ U ] =
  1.0000   2.0000   1.0000   2.0000   2.0000
  0.0000  -1.0000   1.0000  -1.0000   1.0000
  0.0000   0.0000  -3.0000  -3.0000  -2.0000
  0.0000   0.0000   0.0000 -12.0000   8.0000
  0.0000   0.0000   0.0000   0.0000  -5.3333
```

Output of P, L_orig, U at each iteration number :

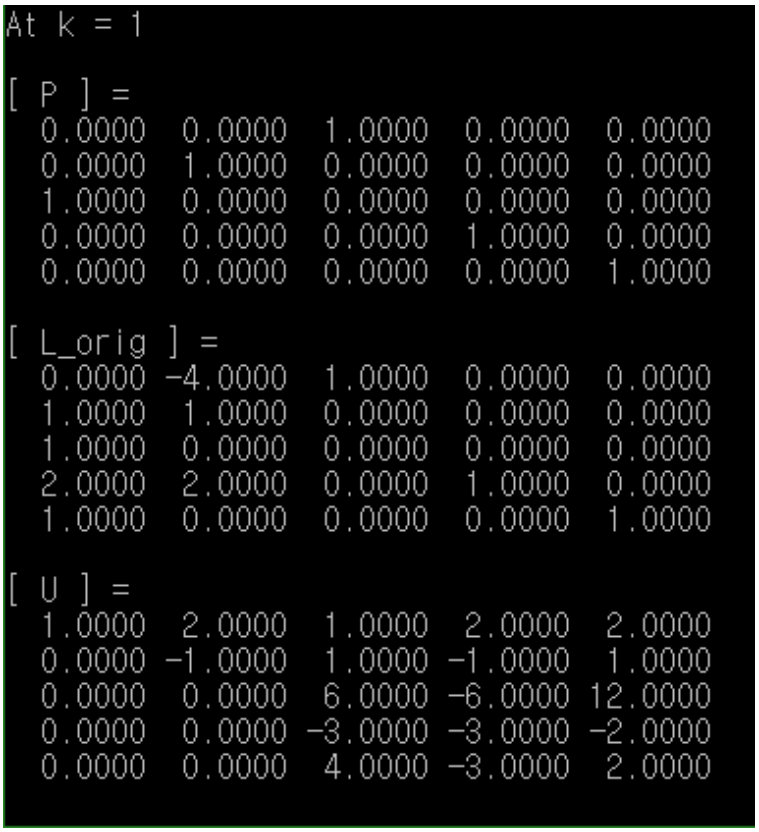


```
At k = 0

[ P ] =
  0.0000  0.0000  1.0000  0.0000  0.0000
  0.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  1.0000  0.0000
  0.0000  0.0000  0.0000  0.0000  1.0000

[ L_orig ] =
  0.0000  0.0000  1.0000  0.0000  0.0000
  1.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  2.0000  0.0000  0.0000  1.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  1.0000

[ U ] =
  1.0000  2.0000  1.0000  2.0000  2.0000
  0.0000 -1.0000  1.0000 -1.0000  1.0000
  0.0000  4.0000  2.0000 -2.0000  8.0000
  0.0000 -2.0000 -1.0000 -5.0000  0.0000
  0.0000  0.0000  4.0000 -3.0000  2.0000
```



```
At k = 1

[ P ] =
  0.0000  0.0000  1.0000  0.0000  0.0000
  0.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  1.0000  0.0000
  0.0000  0.0000  0.0000  0.0000  1.0000

[ L_orig ] =
  0.0000 -4.0000  1.0000  0.0000  0.0000
  1.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  2.0000  2.0000  0.0000  1.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  1.0000

[ U ] =
  1.0000  2.0000  1.0000  2.0000  2.0000
  0.0000 -1.0000  1.0000 -1.0000  1.0000
  0.0000  0.0000  6.0000 -6.0000 12.0000
  0.0000  0.0000 -3.0000 -3.0000 -2.0000
  0.0000  0.0000  4.0000 -3.0000  2.0000
```

```
At k = 2

[ P ] =
  0.0000  0.0000  1.0000  0.0000  0.0000
  0.0000  1.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  1.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  0.0000  1.0000

[ L_orig ] =
  0.0000 -4.0000 -2.0000  1.0000  0.0000
  1.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  2.0000  2.0000  1.0000  0.0000  0.0000
  1.0000  0.0000 -1.3333  0.0000  1.0000

[ U ] =
  1.0000  2.0000  1.0000  2.0000  2.0000
  0.0000 -1.0000  1.0000 -1.0000  1.0000
  0.0000  0.0000 -3.0000 -3.0000 -2.0000
  0.0000  0.0000  0.0000-12.0000  8.0000
  0.0000  0.0000  0.0000 -7.0000 -0.6667
```

```
At k = 3

[ P ] =
  0.0000  0.0000  1.0000  0.0000  0.0000
  0.0000  1.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  1.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  0.0000  1.0000

[ L_orig ] =
  0.0000 -4.0000 -2.0000  1.0000  0.0000
  1.0000  1.0000  0.0000  0.0000  0.0000
  1.0000  0.0000  0.0000  0.0000  0.0000
  2.0000  2.0000  1.0000  0.0000  0.0000
  1.0000  0.0000 -1.3333  0.5833  1.0000

[ U ] =
  1.0000  2.0000  1.0000  2.0000  2.0000
  0.0000 -1.0000  1.0000 -1.0000  1.0000
  0.0000  0.0000 -3.0000 -3.0000 -2.0000
  0.0000  0.0000  0.0000-12.0000  8.0000
  0.0000  0.0000  0.0000  0.0000 -5.3333
```